

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Ордена трудового Красного Знамени федеральное государственное
бюджетное**

образовательное учреждение высшего образования

«Московский технический университет связи и информатики»

Кафедра Математическая кибернетика и информационные технологии

Отчет по лабораторной работе №3.

“Разработка веб-приложения на Spring Boot. Введение в IoC/DI”

Выполнил: студент группы БВТ2402

Скопа Михаил Вячеславович

Москва, 2026

Цель работы: Научиться создавать Spring Boot приложение, освоить принципы инверсии управления (IoC) и внедрения зависимостей (DI), изучить три способа конфигурации Spring компонентов, а также научиться разрешать конфликты при наличии нескольких бинов одного типа.

Задание для выполнения лабораторной работы

1. Добавьте еще один метод в HelloController, который обрабатывает GET-запрос на /goodbye и возвращает "До свидания, Spring Boot!".
2. Добавьте метод, который принимает параметр name через @RequestParam и возвращает "Привет, name!". Например, запрос /greet?name=Иван должен вернуть "Привет, Иван!".
3. (Дополнительное задание) Добавьте метод, который принимает два параметра (name и age) и возвращает сообщение с информацией о пользователе.

```
6
7  @RestController
8  public class HelloController {
9
10     @GetMapping("/hello")
11     public String sayHello() {
12         return "Привет, Spring Boot!";
13     }
14
15     @GetMapping("/goodbye")
16     public String sayGoodbye() {
17         return "До свидания, Spring Boot!";
18     }
19
20
21     @GetMapping("/greet")
22     public String greetUser(@RequestParam String name) {
23         return "Привет, " + name + "!";
24     }
25
26
27     @GetMapping("/info")
28     public String getUserInfo(@RequestParam String name, @RequestParam int age) {
29         return "Пользователь: " + name + ", Возраст: " + age;
30     }
31 }
```

Реализовал метод @GetMapping("/hello"). Он выводит на странице надпись "Привет, Spring Boot!". Метод @GetMapping("/goodbye") выводит на странице надпись "До свидания, Spring Boot!".

В методах с “/greet” и “/info” получение значений параметров name и age реализовано через @RequestParam, который сам находит значение заданного параметра и обрабатывает его.

2

1. Модифицируйте NotificationManager так, чтобы он мог работать с любым сервисом, реализующим MessageService, а не только с EmailService. Измените конструктор и поле соответствующим образом.

2. Создайте третий сервис PushService, который выводит сообщение о push-уведомлении. Как можно использовать его с вашим модифицированным NotificationManager?

3. Ответьте письменно: какие проблемы остаются в коде даже после ваших улучшений? Почему такой подход все еще не удобен для больших проектов?

Файл EmailService.java. Он реализует отправку через почту.

```
public class EmailService implements MessageService {  
    @Override  
    public void sendMessage(String message, String recipient) {  
        System.out.println("EMAIL to " + recipient + ": " + message);  
    }  
}
```

Файл SmsService.java. Он реализует отправку через СМС.

```
public class SmsService implements MessageService {  
    @Override  
    public void sendMessage(String message, String recipient) {  
        System.out.println("SMS to " + recipient + ": " + message);  
    }  
}
```

Файл PushService.java. Реализует отправку уведомлений через push.

```
public class PushService implements MessageService {  
    @Override  
    public void sendMessage(String message, String recipient) {  
        System.out.println("PUSH notification to " + recipient + ": " + message);  
    }  
}
```

Файл MessageService.java. Позволяет менеджеру работать с любым из своих классов наследников. Сам не отправляет уведомления.

```
✓ public interface MessageService {  
    void sendMessage(String message, String recipient);  
}
```

Файл NotificationManager.java. Он реализует вывод уведомления.

```
✓ public class NotificationManager {  
    private final MessageService messageService;  
  
    ✓ public NotificationManager(MessageService messageService) {  
        this.messageService = messageService;  
    }  
  
    ✓ public void notify(String message, String recipient) {  
        messageService.sendMessage(message, recipient);  
    }  
}
```

Файл NotificationController.java. Получает запросы с сайта, связывает их с необходимыми методами и передает дальше.

```
@RestController  
public class NotificationController {  
  
    @GetMapping("/notify")  
    public String notifyUser(@RequestParam String message, @RequestParam String email) {  
        EmailService emailService = new EmailService();  
        NotificationManager manager = new NotificationManager(emailService);  
  
        manager.notify(message, email);  
        return "Уведомление отправлено через Email (жесткая связь)";  
    }  
  
    @GetMapping("/notify-push")  
    public String notifyPush(@RequestParam String message, @RequestParam String deviceId) {  
        PushService pushService = new PushService();  
        NotificationManager manager = new NotificationManager(pushService);  
  
        manager.notify(message, deviceId);  
        return "Уведомление отправлено через Push!";  
    }  
}
```

Какие проблемы остаются в коде даже после улучшений?

Мы пишем new NotificationManager(...) внутри методов контроллера. Это значит, что при каждом запросе создаются новые объекты, которые потом

нужно удалять сборщику мусора, что является не эффективным. Также, если у NotificationManager появится еще 5 зависимостей (например, для логирования, базы данных, проверки прав), то придется переписывать создание этого объекта во всех контроллерах, где он используется.

Почему такой подход все еще неудобен для больших проектов?

В большом проекте могут быть тысячи классов. Если управлять ими вручную через new, то изменение конструктора одного базового сервиса будет требовать переписывать код в сотнях мест по всему проекту; нельзя легко подменить реальный сервис на "заглушку" для тестов, так как создание объекта жестко прописано в коде контроллера.

3

1.Измените метод notificationManager() в AppConfig так, чтобы он внедрял smsService() вместо emailService(). Запустите и убедитесь, что теперь уведомления отправляются через SMS.

```
@Bean
public NotificationManager notificationManager() {
    return new NotificationManager(smsService());
}
```

Для переключения с email SMS достаточно изменить одну строчку в файле конфигурации, что очень удобно.

Было `EMAIL to test@test.com: JavaConfig`

Стало `SMS to test@test.com: JavaConfig`

2.Добавьте в AppConfig создание бина для PushService (который вы создали в предыдущей части). Измените notificationManager() так, чтобы он использовал новый сервис.

Добавил создание бина для PushService.

```
@Bean
public PushService pushService() {
    return new PushService();
}

@Bean
public NotificationManager notificationManager() {
    return new NotificationManager(pushService());
}
```

```
PUSH notification to test@test.com: JavaConfig
□
```

3. (Дополнительное задание) Создайте еще один конфигурационный класс AnotherConfig, импортируйте его в главный класс через `@Import({AppConfig.class, AnotherConfig.class})` Перенесите создание одного из сервисов в новый конфигурационный класс. Убедитесь, что приложение продолжает работать.

Создал новый файл AnotherConfig. Добавил в него бин для создания sms уведомлений.

```
@Configuration
public class AnotherConfig {

    @Bean
    public SmsService smsService() {
        return new SmsService();
    }
}
```

Импортировал его в главный класс и запустил программу. Из файла AppConfig бин для создания sms удалил.

```

@SpringBootApplication
@Import({AppConfig.class, AnotherConfig.class})
public class SpringLab3NotificationsApplication {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        SpringApplication.run(SpringLab3NotificationsApplication.class, args);
    }
}

@Bean
public NotificationManager notificationManager(MessageService smsService) {
    return new NotificationManager(smsService);
}
}

```

Открыв ссылку

<http://localhost:8080/notify?message=TestSMS&email=79991234567> получил:

SMS to 79991234567: TestSMS

4

1. Проанализируйте почему возникла ошибка.

Ошибка возникла, потому что Spring не получил четких указаний какой из трёх сервисов применять.

2. Временно удалите аннотацию `@Service` с одного из классов-сервисов. Запустите приложение снова. Что изменилось? Почему?

Если удалить лишние `@Service` и оставить только один, то проект будет работать с тем сервисом, который остался. Так происходит, потому что больше у `Service` нет сомнений, какой именно сервис использовать.

3. Добавьте в `AppConfig` аннотацию `@ComponentScan` с другим пакетом (например, `"org.example.controller"`). Что произойдет с бинами из пакета `service`? Почему?

Мой проект запустился и работал нормально, так как в главном файле есть аннотация `@SpringBootApplication`, которая имеет у себя свой `@ComponentScan`, который проверяет всё, в текущем пакете. Если изменить на `@SpringBootApplication(scanBasePackages = "org.example.config")`, то будет ошибка, так как проект будет видеть только папку `config`.

4. (Дополнительное задание) Создайте интерфейс `AdvancedMessageService`, который расширяет `MessageService` и добавляет

метод `String getServiceType()`. Реализуйте его в одном из сервисов. Как это повлияет на процесс внедрения зависимости

Создал интерфейс

```
1 package org.example.service;
2
3 public interface AdvancedMessageService extends MessageService {
4     String getServiceType();
5 }
```

Реализовал его в сервисе email

```
@Service
public class EmailService implements AdvancedMessageService {
    @Override
    public void sendMessage(String message, String recipient) {
        System.out.println("ADVANCED EMAIL to " + recipient + ": " + message);
    }

    @Override
    public String getServiceType() {
        return "EMAIL_PROTOCOL";
    }
}
```

Поменял тип принимаемого аргумента в NotificationManager

```
@Service
public class NotificationManager {
    private final AdvancedMessageService messageService;

    @Autowired
    public NotificationManager(AdvancedMessageService messageService) {
        this.messageService = messageService;
    }

    public void notify(String message, String recipient) {
        System.out.println("Using service type: " + messageService.getServiceType());
        messageService.sendMessage(message, recipient);
    }
}
```

```
Using service type: EMAIL_PROTOCOL
ADVANCED EMAIL to 79991234567: TestSMS
```

Это решило проблему с Service, так как теперь только один сервис удовлетворяет необходимому класс принимаемого аргумента.

Самостоятельное задание к Части 5

1. `@Primary`: Сделайте так, чтобы основным стал `SmsService`, используя только `@Primary` (не удаляя и не изменяя другие аннотации).

```
@Service
@Primary
public class SmsService implements MessageService {
    @Override
    public void sendMessage(String message, String recipient) {
        System.out.println("SMS to " + recipient + ": " + message);
    }
}
```

```
SMS to myemail@test.com: HelloSpring
```

2. `@Qualifier`: Измените имя бина для `EmailService` через `@Service("customEmail")`. Исправьте `@Qualifier` в конструкторе, чтобы он использовал новое имя. Что произойдет, если в `@Qualifier` указать несуществующее имя?

```
@Autowired
public NotificationManager(@Qualifier("smsService") MessageService messageService) {
    this.messageService = messageService;
}
public void notify(String message, String recipient) {
    messageService.sendMessage(message, recipient);
}
```

```
SMS to myemail@test.com: HelloSpring
```

Если указать несуществующее имя, то выйдет ошибка, так как `Service` не нашел подходящего сервиса.

2. Коллекция: Добавьте четвертый сервис (например, `TelegramService`) и убедитесь, что он автоматически попадает в список без изменения кода `NotificationManager`. Почему это работает?

```
@Service
public class NotificationManager {
    private final List<MessageService> messageServices;
    @Autowired
    public NotificationManager(List<MessageService> messageServices)
    {
        this.messageServices = messageServices;
    }
    public void notify(String message, String recipient) {
        messageServices.forEach(service -> service.sendMessage(message, recipient));
    }
}
```

```

@Service
public class TelegramService implements MessageService {
    @Override
    public void sendMessage(String message, String recipient) {
        System.out.println("Telegram to " + recipient + ": " + message);
    }
}

```

```

EMAIL to myemail@test.com: HelloSpring
PUSH notification to myemail@test.com: HelloSpring
SMS to myemail@test.com: HelloSpring
Telegram to myemail@test.com: HelloSpring

```

Создал еще один сервис, предварительно внедрив коллекцию в Notification. Коллекции – это самый гибкий способ и он выводит все доступные сервисы одновременно.

4. Комбинирование: Попробуйте одновременно использовать @Primary на одном сервисе и внедрение List в другом месте. Как это работает?

```

EMAIL to myemail@test.com: HelloSpring
PUSH notification to myemail@test.com: HelloSpring
SMS to myemail@test.com: HelloSpring
Telegram to myemail@test.com: HelloSpring

```

Добавил @Primary в сервис sms, но ничего не поменялось, так как коллекции игнорируют Primary.

5. (Дополнительное задание) Внедрите Map вместо List: Измените метод notify так, чтобы он отправлял уведомления только определенными типами сервисов на основе переданного параметра

```

@Service
public class NotificationManager {
    private final Map<String, MessageService> serviceMap;

    public NotificationManager(Map<String, MessageService> serviceMap) {
        this.serviceMap = serviceMap;
    }

    public void notify(String type, String message, String recipient) {
        MessageService service = serviceMap.get(type);

        if (service != null) {
            service.sendMessage(message, recipient);
        } else {
            System.out.println("Сервис типа " + type + " не найден! Доступны: " + serviceMap.keySet());
        }
    }
}

```

```

@RestController
public class NotificationController {

    private final NotificationManager notificationManager;

    public NotificationController(NotificationManager notificationManager) {
        this.notificationManager = notificationManager;
    }

    @GetMapping("/notify")
    public String notify(
        @RequestParam String message,
        @RequestParam String email,
        @RequestParam String type
    ) {
        notificationManager.notify(type, message, email);
        return "Запрос на отправку через " + type + " принят";
    }
}

```

Теперь, при параметрах
<http://localhost:8080/notify?message=Hi&email=123&type=smsService> в
 консоли будет **SMS to 123: Hi**, а при
<http://localhost:8080/notify?message=Hi&email=test@mail.ru&type=emailService>
EMAIL to test@mail.ru: Hi

Контрольные вопросы

1. В чем разница между `@Component`, `@Service`, `@Repository` и `@Controller`?

@Component: Базовая аннотация для любого бина.

@Service: Для классов с бизнес-логикой.

@Repository: Для работы с базами данных (Spring дополнительно включает перехват исключений БД).

@Controller / @RestController: Для обработки HTTP-запросов.

2. Как работает `@ComponentScan`? Что произойдет, если указать неправильный пакет?

`@ComponentScan` заставляет Spring сканировать указанный пакет и все его подпакеты в поиске классов с аннотациями `@Component` (и производными).

Spring просто не «увидит» классы. В результате возникнет ошибка `NoSuchBeanDefinitionException`, так как зависимости не будут найдены и внедрены.

3. Почему в Части 3 не возникло ошибки с двумя бинами, а в Части 4 возникла?

В Части 3 использовалась явная конфигурацию (`JavaConfig` с методами `@Bean`). Вручную указывалось, какой объект создать, поэтому выбора у Spring не было.

В Части 4 действовала автоматика. Spring увидел сразу три подходящих класса (`Email`, `Sms`, `Push`) и не смог выбрать один без точного указания.

4. Какие способы разрешения конфликта бинов вы знаете? Какой самый гибкий?

`@Primary`: Помечает один бин как «основной» по умолчанию.

`@Qualifier("name")`: Точечное указание имени бина прямо в месте внедрения.

Самый гибкий `List`: выводит список всех доступных сервисов.

`Map`: выводит список необходимых сервисов.

5. Как внедрить все бины определенного типа в коллекцию или карту?

Для этого можно использовать `List<>`. Он выведет список всех доступных сервисов. (Указать `List<>` или `Map<>` в качестве аргумента конструктора с `@Autowired`)

6. В чем преимущества внедрения через конструктор перед внедрением через поле?

Неизменяемость: Можно использовать ключевое слово `final` для зависимостей.

Тестируемость: Класс легко тестировать в Unit-тестах, просто передавая зависимости в конструктор вручную (не нужен Spring).

Безопасность: Нельзя создать объект в «неполном» состоянии — Java просто не даст скомпилировать код без передачи зависимостей.

Отсутствие скрытых зависимостей: Сразу видно, сколько всего нужно классу для работы.

Вывод: Научился создавать Spring Boot приложение, освоил принципы инверсии управления (IoC) и внедрения зависимостей (DI), изучил три способа конфигурации Spring компонентов, а также научился разрешать конфликты при наличии нескольких бинов одного типа.

Ссылка на git: <https://github.com/Mirphon/ITIP/tree/main/Lab10>